پارسا عربشاهی 402521405

پیاده‌سازی سیستم فراخوانی Checkpoint/Restore درxv6-riscv

هدف پروژه

هدف این پروژه افزودن قابلیت **Checkpoint/Restore** به سیستم‌عامل آموزشی xv6-riscv است. این قابلیت به یک پردازه اجازه می‌دهد وضعیت اجرای خود را ذخیره کرده و در آینده اجرای خود را دقیقاً از همان نقطه ادامه دهد.

این قابلیت مشابه قابلیت **Save State** درEmulator ها و Snapshot در ماشین‌های مجازی است.

قابلیت‌های پیاده‌سازی شده

دو System Call جدید به xv6 اضافه شده است.

```
int checkpoint(int id);
int restore(int id);
```

هر پردازه می‌تواند حداکثر چهار Checkpoint داشته باشد.

```
checkpoint(0)
checkpoint(1)
checkpoint(2)
checkpoint(3)
```

اطلاعات ذخیره‌شده

در هر Checkpoint اطلاعات زیر ذخیره می‌شود.

۱ -وضعیت ثبات‌های پردازنده

تمام محتویات Trapframe ذخیره می‌شود.

```
ra
sp
gp
tp
a0-a7
s0-s11
t0-t6
epc
...
```

به همین دلیل پس از Restore برنامه دقیقاً از همان دستور قبلی ادامه پیدا می‌کند.

---

۲ -فضای آدرس پردازه

تمام حافظه کاربر کپی می‌شود.

Code
Data
Heap
Stack

برای این منظور از

uvmclone()

استفاده شده است.

درون این تابع، Page Table جدید ساخته شده و تمام صفحات فیزیکی حافظه کاربر کپی می‌شوند.

---

۳ -اندازه حافظه

p->sz

نیز ذخیره می‌شود تا هنگام Restore اندازه حافظه کاملاً مشابه قبل باشد.

---

۴ -فایل‌های باز

تمام File Descriptor ها با

filedup()

ذخیره می‌شوند.

در نتیجه پس از Restore همان فایل‌ها همچنان باز هستند.

۵- Current Working Directory

دایرکتوری جاری پردازه نیز با

idup()

ذخیره و بازیابی می‌شود.

ساختار داده Checkpoint

```
struct checkpoint {
   int used;

   struct trapframe tf;

   uint64 sz;

   pagetable_t pagetable;

   struct file *ofile[NOFILE];

   struct inode *cwd;
};
```

هر پردازه شامل

```
struct checkpoint cps[4];
```

است.

روند اجرای checkpoint()

هنگام فراخوانی

```
checkpoint(id);
```

مراحل زیر انجام می‌شود.

Process

↓

Save Trapframe

↓

Duplicate Open Files

↓

Duplicate CWD

↓

Clone User Memory

↓

Store Checkpoint

روند اجرای ()restore

Restore(id)

↓

Free Current Address Space

↓

Replace Page Table

↓

Restore Trapframe

↓

Restore Open Files

↓

Restore CWD

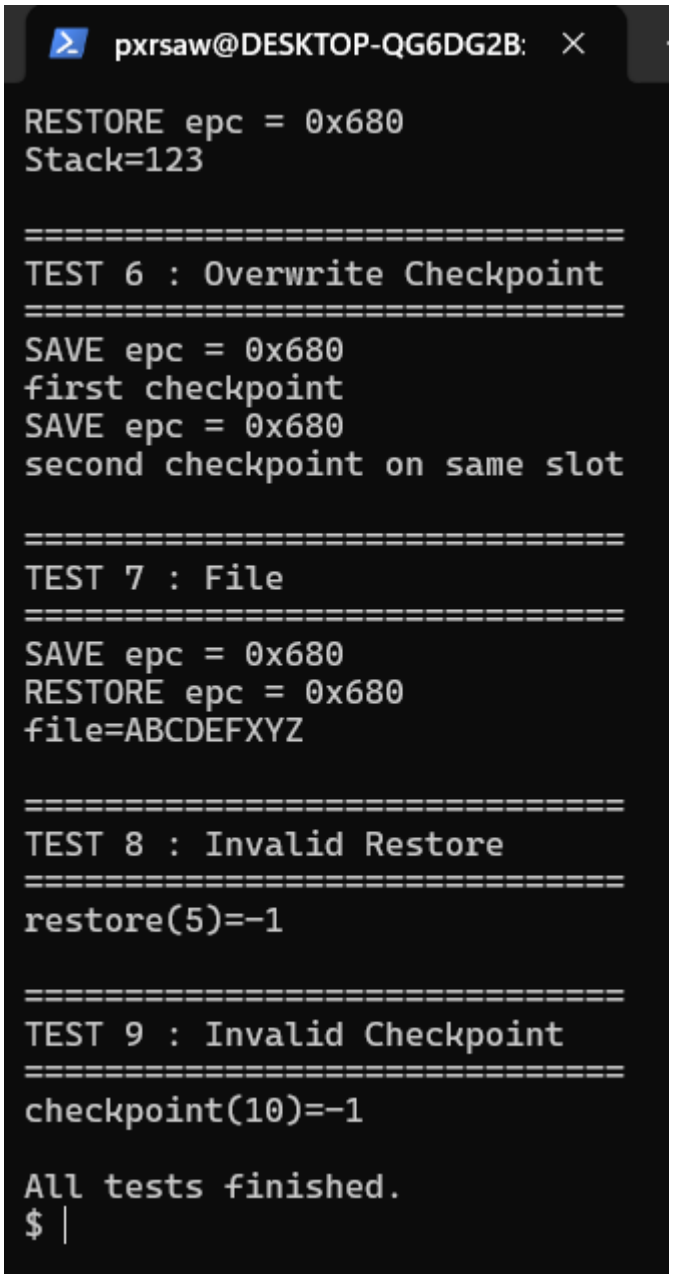
↓

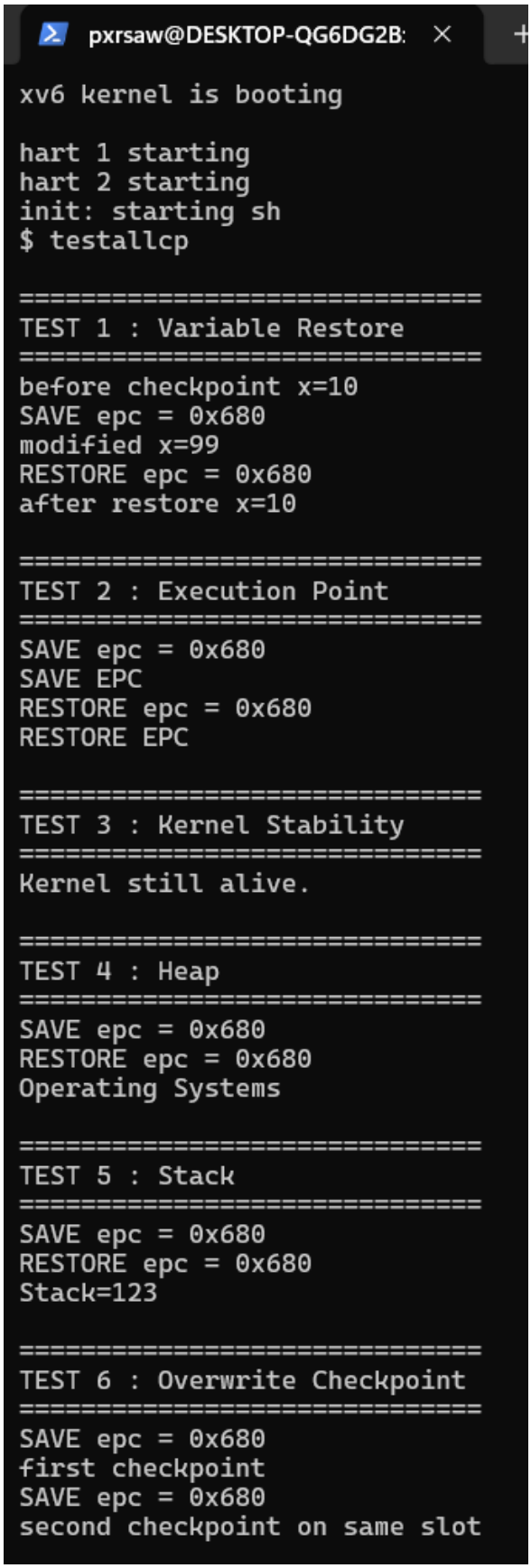
Continue Execution

به همین دلیل برنامه دقیقاً از همان دستور قبلی ادامه پیدا می‌کند.

تست‌های انجام شده

در فایل testallcp.c 9 تست نوشته شده که نتایج آن ها به صورت زیر می‌باشد


pxrsaw@DESKTOP-QG6DG2B:
RESTORE epc = 0x680
Stack=123
==========
TEST 6 : Overwrite Checkpoint
==========
SAVE epc = 0x680
first checkpoint
SAVE epc = 0x680
second checkpoint on same slot
==========
TEST 7 : File
==========
SAVE epc = 0x680
RESTORE epc = 0x680
file=ABCDEFXYZ
==========
TEST 8 : Invalid Restore
==========
restore(5)=-1
==========
TEST 9 : Invalid Checkpoint
==========
checkpoint(10)=-1
All tests finished.
$


pxrsaw@DESKTOP-QG6DG2B:
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ testallcp
==========
TEST 1 : Variable Restore
==========
before checkpoint x=10
SAVE epc = 0x680
modified x=99
RESTORE epc = 0x680
after restore x=10
==========
TEST 2 : Execution Point
==========
SAVE epc = 0x680
SAVE EPC
RESTORE epc = 0x680
RESTORE EPC
==========
TEST 3 : Kernel Stability
==========
Kernel still alive.
==========
TEST 4 : Heap
==========
SAVE epc = 0x680
RESTORE epc = 0x680
Operating Systems
==========
TEST 5 : Stack
==========
SAVE epc = 0x680
RESTORE epc = 0x680
Stack=123
==========
TEST 6 : Overwrite Checkpoint
==========
SAVE epc = 0x680
first checkpoint
SAVE epc = 0x680
second checkpoint on same slot